

7 · Bayesian parameter estimation with Turing.jl ⇄

`04_parameter_estimation.jl` found a single *best-fit* point for the SIR model's parameters. That answers "what values fit best?" but not "how sure are we?" — with noisy, limited data, many nearby parameter values could explain the data almost as well.

Bayesian inference answers the second question directly: instead of a point estimate, it returns a full *posterior distribution* over the parameters, obtained by combining a prior belief with the likelihood of the observed data under the ODE model. We sample that posterior with Turing.jl's No-U-Turn Sampler (NUTS), reusing exactly the same `OrdinaryDiffEq.jl` forward model as the point-estimate notebook — only the fitting method changes.

```
1 using Turing, OrdinaryDiffEq, Random, Statistics, Plots
```

1. The model and synthetic data ⇄

Same SIR epidemic model and the same synthetic noisy observations of $I(t)$ as in `04_parameter_estimation.jl`.

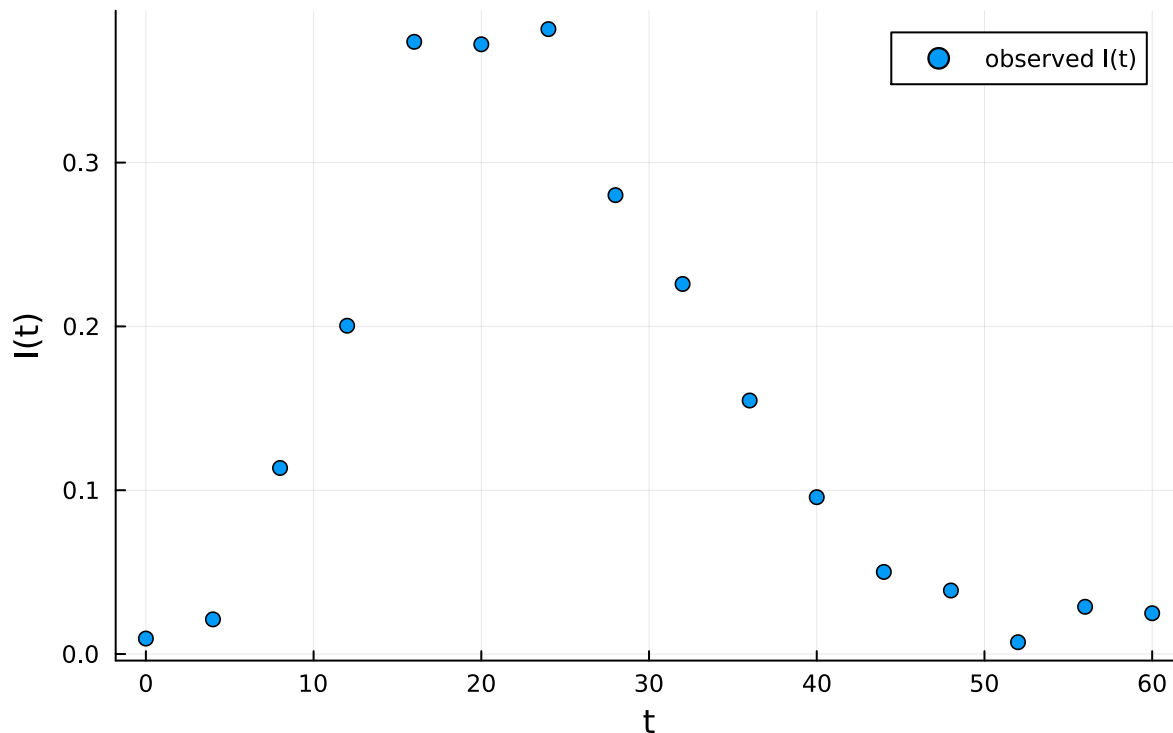
`sir!` (generic function with 1 method)

```
1 function sir!(du, u, p, t)
2     S, I, R = u
3     β, γ = p
4     du[1] = -β * S * I
5     du[2] = β * S * I - γ * I
6     du[3] = γ * I
7     return nothing
8 end
```

```
► [0.00944524, 0.0211624, 0.113578, 0.200418, 0.373723, 0.372206, 0.381471, 0.280126, 0.225902,
```

```
1 begin
2   rng7 = Random.default_rng()
3   Random.seed!(rng7, 21)
4
5   true_β7, true_γ7 = 0.4, 0.1
6   u0_sir7 = [0.99, 0.01, 0.0]
7   tspan_sir7 = (0.0, 60.0)
8   tsteps_sir7 = range(tspan_sir7[1], tspan_sir7[2]; length=16)
9
10  sir_prob7 = ODEProblem(sir!, u0_sir7, tspan_sir7, [true_β7, true_γ7])
11  sir_sol7 = solve(sir_prob7, Tsit5(); saveat=tsteps_sir7)
12
13  noise_level7 = 0.02
14  observed_I = Array(sir_sol7)[2, :] .+ noise_level7 .* randn(rng7,
15  length(tsteps_sir7))
16 end
```

Data given to the Bayesian model



```
1 scatter(tsteps_sir7, observed_I; label="observed I(t)", xlabel="t", ylabel="I(t)",
2 title="Data given to the Bayesian model")
```

2. The Bayesian model ⇌

@model declares priors on β , γ , and the observation noise σ , solves the ODE for a candidate parameter draw, and scores how likely the observed $I(t)$ is under a normal observation model centered on the simulated trajectory. Turing.jl differentiates straight through the OrdinaryDiffEq.jl solve (forward-mode, the same mechanism used for the fits in the O&G notebooks) to drive NUTS.

```

sir_bayesian (generic function with 2 methods)

1 Turing.@model function sir_bayesian(data, prob, tsteps)
2    $\beta \sim \text{truncated}(\text{Normal}(0.5, 0.5); \text{lower}=0.0, \text{upper}=3.0)$ 
3    $\gamma \sim \text{truncated}(\text{Normal}(0.2, 0.2); \text{lower}=0.0, \text{upper}=3.0)$ 
4    $\sigma \sim \text{truncated}(\text{Normal}(0.05, 0.05); \text{lower}=0.001, \text{upper}=1.0)$ 
5
6   sol = solve(prob, Tsit5(); p=[ $\beta$ ,  $\gamma$ ], saveat=tsteps)
7   if sol.retcode != ReturnCode.Success
8       Turing.@addlogprob! -Inf
9       return nothing
10  end
11
12  predicted_I = Array(sol)[2, :]
13  for i in eachindex(tsteps)
14      data[i] ~ Normal(predicted_I[i],  $\sigma$ )
15  end
16  return nothing
17 end

```

3. Sample the posterior ⇌

```

FlexiChain (1000 iterations, 1 chain)
↓ iter = 501:1500
→ chain = 1:1

Parameters (3) — AbstractPPL.VarName
Float64  $\beta$ ,  $\gamma$ ,  $\sigma$ 

Extras (14)
Int64 n_steps, tree_depth
Bool is_accept, numerical_error
Float64 acceptance_rate, log_density, hamiltonian_energy,
hamiltonian_energy_error, max_hamiltonian_energy_error, step_size,
nom_step_size, logprior, loglikelihood, logjoint

```

```

1 begin
2   bayes_model = sir_bayesian(observed_I, sir_prob7, tsteps_sir7)
3   chain = sample(bayes_model, NUTS(0.65), 1000)
4 end

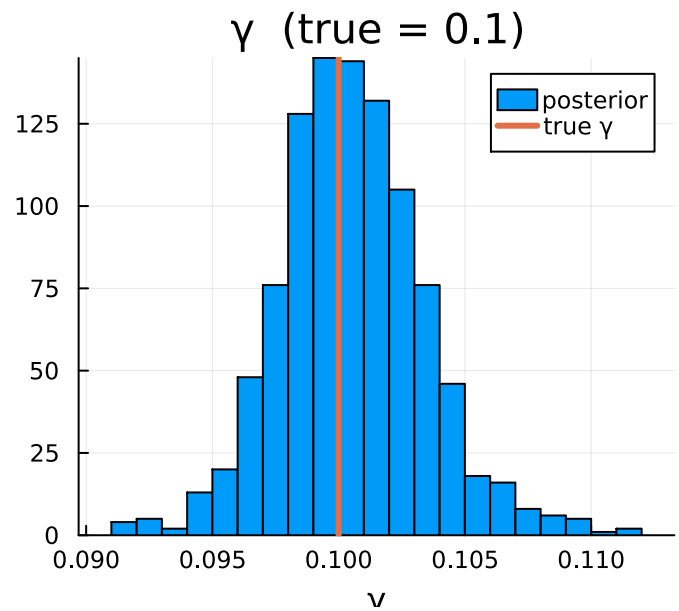
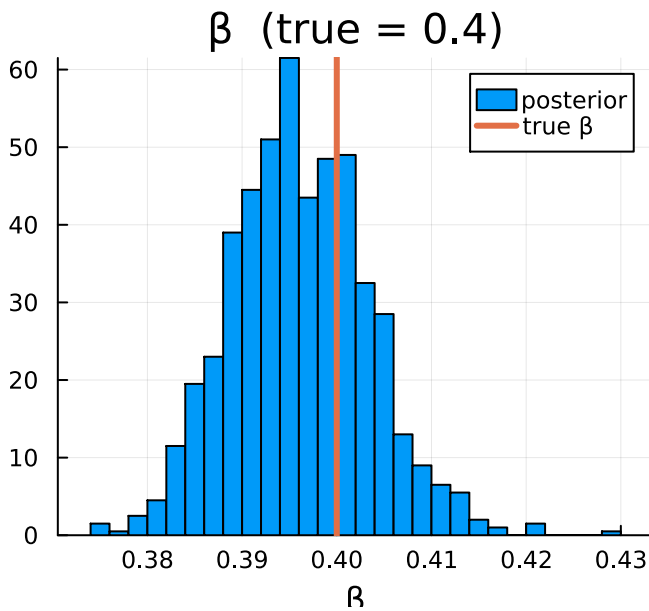
```

Sampling 100%

① Found initial step size
 ϵ : 0.4

4. Posterior distributions vs. the true values ⇌

Unlike the single point estimate from `Optimization.jl`, NUTS returns a full sample from the posterior — its spread is the uncertainty.



```

1 begin
2   beta_samples = vec(Array(chain[:beta]))
3   gamma_samples = vec(Array(chain[:gamma]))
4
5   p1 = histogram(beta_samples; label="posterior", xlabel="beta", title="beta (true =
6   $(true_beta)", normalize=true)
7   vline!(p1, [true_beta]; label="true beta", lw=3)
8
9   p2 = histogram(gamma_samples; label="posterior", xlabel="gamma", title="gamma (true =
10  $(true_gamma)", normalize=true)
11  vline!(p2, [true_gamma]; label="true gamma", lw=3)
12  plot(p1, p2; layout=(1, 2), size=(700, 300))
13 end

```

► ($\beta_{\text{mean}} = 0.396065$, $\beta_{\text{std}} = 0.00731124$, $\gamma_{\text{mean}} = 0.100526$, $\gamma_{\text{std}} = 0.00287584$)

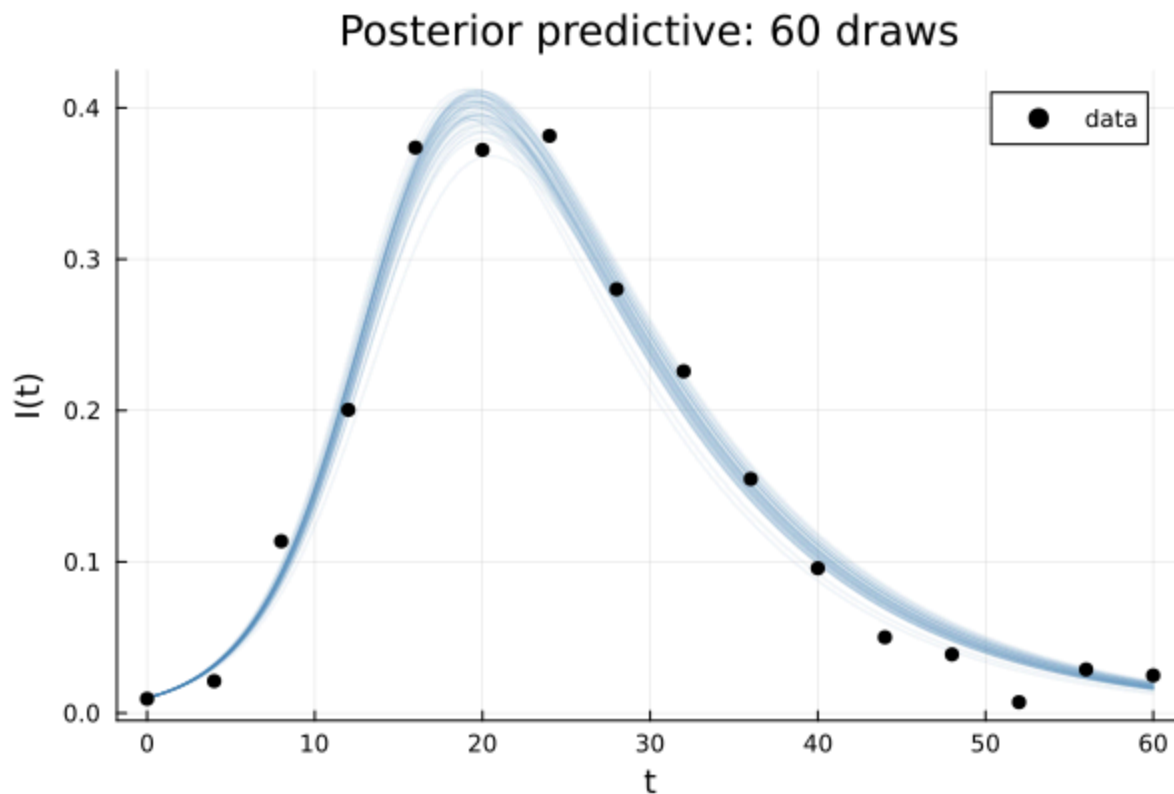
```

1 (beta_mean=mean(beta_samples), beta_std=std(beta_samples), gamma_mean=mean(gamma_samples),
  gamma_std=std(gamma_samples))

```

5. Posterior predictive check ⇔

Solving the ODE forward for many posterior draws (instead of just the best fit) gives a *band* of plausible trajectories — a direct visualization of forecast uncertainty.



```

1 begin
2     n_draws = 60
3     draw_idx = rand(rng7, 1:length(β_samples), n_draws)
4     t_fine = range(tspan_sir7[1], tspan_sir7[2]; length=200)
5
6     plt = plot(; xlabel="t", ylabel="I(t)", title="Posterior predictive: $(n_draws)
7         draws", legend=:topright)
8     for i in draw_idx
9         draw_sol = solve(remake(sir_prob7; p=[β_samples[i], y_samples[i]]), Tsit5();
10             saveat=t_fine)
11         plot!(plt, t_fine, Array(draw_sol)[2, :]; color=:steelblue, alpha=0.08,
12             label=false)
13     end
14     scatter!(plt, tsteps_sir7, observed_I; color=:black, label="data")
15     plt
16 end

```

Takeaways ⇄

- A Bayesian model is the same forward simulation as before, wrapped in priors and a likelihood — Turing.jl handles the sampling machinery, not the ODE.
- The posterior's spread is honest uncertainty: it widens automatically with noisier or sparser data, something a single point estimate from [04_parameter_estimation.jl](#) cannot express on its own.
- The posterior *predictive* band (many forward solves, one per posterior draw) is the natural way to report a forecast with uncertainty, rather than a single curve.

